

제품 요구사항 문서 (PRD)

제품 요구사항 문서 (PRD)

0. 요약

1. 시스템 아키텍처

1.1 상위 파이프라인 흐름

1.2 기술 스택

1.3 목표 디렉터리 구조

2. 구현 단계

1단계: 환경 설정 및 장면 재구성

2단계: Grounded SAM 2를 이용한 의미론적 분할

3단계: SDS 기반 객체 생성 및 배치

4단계: PhysGaussian을 이용한 물리 시뮬레이션

5단계: 평가 및 최종 산출물

3. 전역 설정 스키마

4. Cursor AI를 위한 구현 시 주의사항

4.1 VRAM 관리 전략

4.2 서브모듈 간 PLY 호환성

4.3 좌표계 정렬

4.4 PhysGaussian 장면 준비

5. 테스트 및 검증 체크리스트

6. 산출물 요약

제품 요구사항 문서 (PRD)

프로젝트: 대화형 효과가 있는 텍스트 유도 3D 실내 편집기

작성: 방민재 (Columbia University, SEAS)

대상 독자: Cursor AI (AI 코딩 어시스턴트)

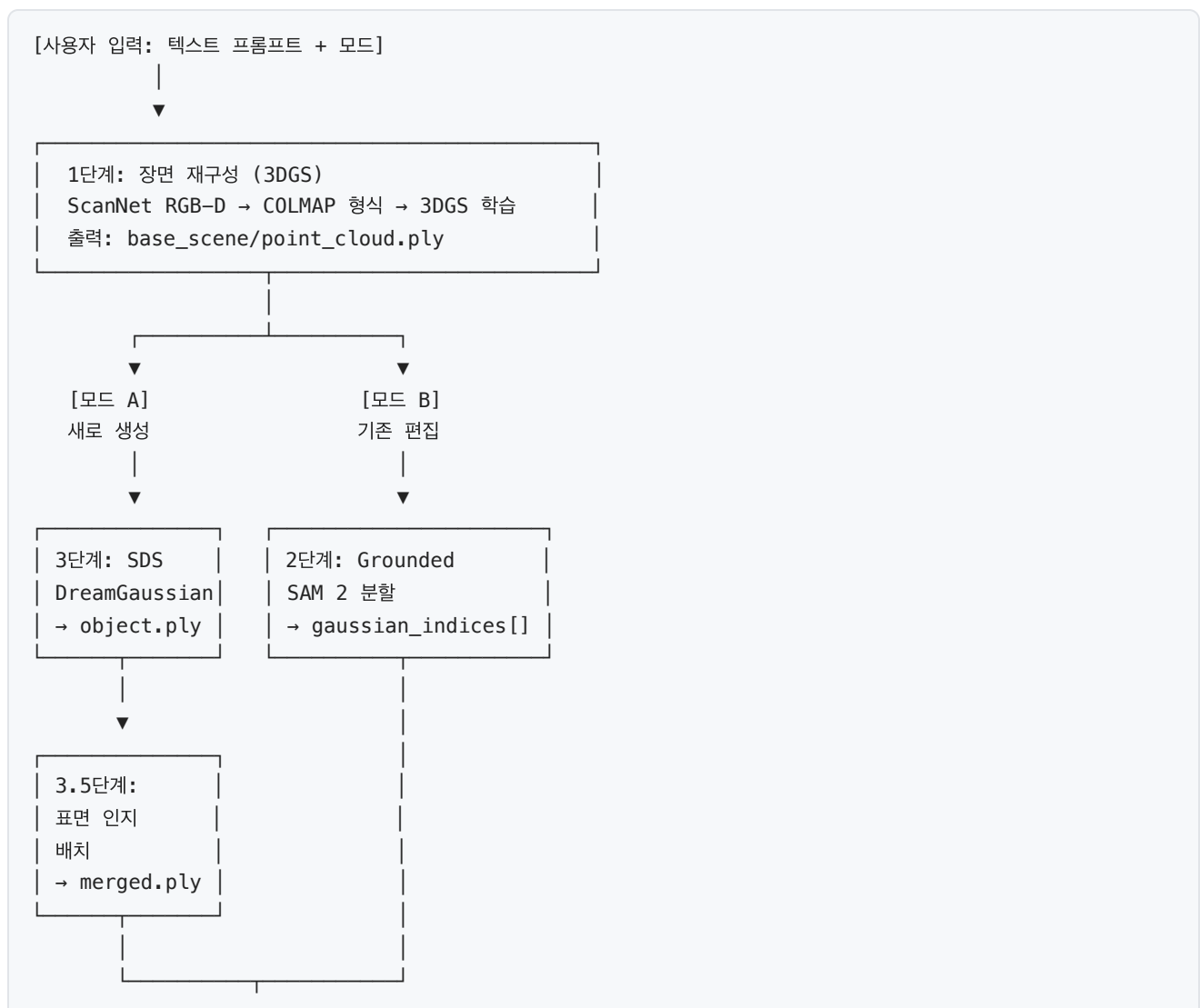
목적: 본 문서는 텍스트 유도 3D 실내 장면 편집 시스템을 구현하기 위한 단계별 설계 청사진을 제공합니다. Cursor AI는 각 단계를 순서대로 따르고, 다음 단계로 넘어가기 전에 모든 작업을 완료하고 검증 기준을 통과해야 합니다.

0. 요약

본 프로젝트는 사용자가 (1) 텍스트 프롬프트로 완전히 새로운 3D 객체를 생성해 기존 3D Gaussian Splatting(3DGS) 장면에 삽입하고, (2) 텍스트로 장면 안의 기존 객체를 선택한 뒤 녹는 것, 부풀리는 것, 젤리 같은 흔들림 등 물리 기반 효과를 적용할 수 있는 통합 파이프라인을 만듭니다. 파이프라인은 네 가지 주요 오픈소스 구성요소를 연결합니다: 장면 표현을 위한 3D Gaussian Splatting, 텍스트 기반 분할을 위한 Grounded SAM 2, SDS 기반 객체 생성을 위한 DreamGaussian, MPM 기반 물리 시뮬레이션을 위한 PhysGaussian.

1. 시스템 아키텍처

1.1 상위 파이프라인 흐름





1.2 기술 스택

구성요소	저장소	역할	주요 파일
3DGS	graphdeco-inria/gaussian-splatting	장면 재구성 및 렌더링	train.py, render.py
Grounded SAM 2	IDEA-Research/Grounded-SAM-2	텍스트 → 바운딩 박스 → 분할 마스크	grounded_sam2_local_demo.py
DreamGaussian	dreamgaussian/dreamgaussian	SDS로 텍스트 → 3D Gaussian 객체	main.py, guidance/sd_utils.py
PhysGaussian	XPandora/PhysGaussian	Gaussian 위 MPM 물리 시뮬레이션	gs_simulation.py, mpm_solver_warp/
Stable Diffusion	HuggingFace (stabilityai/stable-diffusion-2-1-base)	SDS 손실용 2D 확산 사전	diffusers로 로드

1.3 목표 디렉터리 구조

```
text-guided-3d-editor/
├─ setup.sh                # 원클릭 환경 설정
├─ requirements.txt
├─ README.md
├─
├─ data/
├─   └─ scannet_raw/      # 원본 ScanNet 장면(별도 다운로드)
```

```

├── scene0000_00/
│   ├── color/           # RGB 이미지
│   ├── depth/           # 깊이 맵
│   ├── pose/            # 카메라 외부 파라미터 (4x4 .txt)
│   └── intrinsic/       # 카메라 내부 파라미터
├── scannet_colmap/      # 변환된 COLMAP 형식(생성됨)
│   ├── scene0000_00/
│   │   ├── images/
│   │   └── sparse/0/
│   │       ├── cameras.bin
│   │       ├── images.bin
│   │       └── points3D.bin
├── submodules/          # Git 서브모듈(내부 수정 금지)
│   ├── gaussian-splatting/
│   ├── Grounded-SAM-2/
│   ├── dreamgaussian/
│   └── PhysGaussian/
├── output/
│   ├── base_scene/      # 학습된 3DGS 모델
│   │   └── point_cloud/iteration_30000/point_cloud.ply
│   ├── generated_objects/ # SDS로 생성된 객체
│   ├── merged_scenes/   # 배치 후
│   ├── sim_results/     # PhysGaussian 출력 프레임 및 비디오
│   └── eval/            # 정량 평가 결과
├── src/
│   ├── __init__.py
│   ├── pipeline.py      # 메인 오케스트레이터
│   ├── config.py        # Pydantic 설정 모델
│   ├── data_utils/
│   │   ├── __init__.py
│   │   ├── scannet_to_colmap.py # ScanNet → COLMAP 변환기
│   │   └── scannet_downloader.py # ScanNet 장면 다운로드 헬퍼
│   ├── reconstruction/
│   │   ├── __init__.py
│   │   ├── train_3dgs.py      # 3DGS 학습 래퍼
│   │   └── render_views.py   # 학습된 3DGS에서 이미지 렌더
│   ├── segmentation/
│   │   ├── __init__.py
│   │   ├── text_to_mask.py    # Grounding DINO + SAM 2 래퍼
│   │   ├── mask_to_gaussians.py # 2D 마스크 → 3D Gaussian 인덱스
│   │   └── multi_view_consensus.py # 견고성을 위한 다시점 마스크 투표
│   ├── generation/
│   │   ├── __init__.py
│   │   └── sds_generator.py   # DreamGaussian 래퍼

```

```

├── object_rescaler.py      # 생성 객체를 장면 단위로 스케일
├── placement/
│   ├── __init__.py
│   ├── surface_aware.py    # 깊이 기반 표면 검출
│   └── ply_merger.py        # SH 정렬로 두 PLY 병합
├── physics/
│   ├── __init__.py
│   ├── config_generator.py  # PhysGaussian JSON 설정 생성
│   ├── mpm_runner.py        # gs_simulation.py 실행
│   └── material_presets.py  # 미리 정의된 재질 파라미터
├── evaluation/
│   ├── __init__.py
│   ├── metrics.py           # PSNR, SSIM, LPIPS 계산
│   └── ablation.py          # SAM 2 vs LERF 비교
├── configs/
│   ├── pipeline_config.yaml # 전역 파이프라인 설정
│   └── phys_templates/
│       ├── jelly.json
│       ├── metal.json
│       ├── sand.json
│       └── foam.json
├── scripts/
│   ├── run_mode_a.sh        # 모드 A 종단간 데모
│   ├── run_mode_b.sh        # 모드 B 종단간 데모
│   └── run_evaluation.sh     # 정량 평가 실행
└── tests/
    ├── test_scannet_converter.py
    ├── test_segmentation.py
    ├── test_sds_generation.py
    ├── test_placement.py
    └── test_physics.py

```

2. 구현 단계

1단계: 환경 설정 및 장면 재구성

목표: 개발 환경을 구축하고, ScanNet 데이터를 변환하며, 기본 3DGS 장면을 학습한다.

기간: 1-2주

작업 1.1: 환경 설정 스크립트 (setup.sh)

전체 환경 설정을 자동화하는 bash 스크립트를 작성한다. 아래 순서를 반드시 따른다:

```
#!/bin/bash
set -e

# 1. conda 환경 생성
conda create -n room-editor python=3.10 -y
conda activate room-editor

# 2. CUDA용 PyTorch 설치
pip install torch==2.3.1 torchvision==0.18.1 torchaudio==2.3.1 --index-url
https://download.pytorch.org/whl/cu118

# 3. 서브모듈 클론
git submodule update --init --recursive

# 4. 3DGS 의존성 설치
pip install -e submodules/gaussian-splatting/submodules/diff-gaussian-rasterization/
pip install -e submodules/gaussian-splatting/submodules/simple-knn/

# 5. Grounded SAM 2 설치
cd submodules/Grounded-SAM-2
pip install -e .
pip install --no-build-isolation -e grounding_dino
cd checkpoints && bash download_ckpts.sh && cd ..
cd gdino_checkpoints && bash download_ckpts.sh && cd ../..

# 6. DreamGaussian 의존성 설치
pip install -r submodules/dreamgaussian/requirements.txt

# 7. PhysGaussian 의존성 설치
pip install -r submodules/PhysGaussian/requirements.txt

# 8. 프로젝트 수준 의존성 설치
pip install plyfile trimesh lpips scikit-image pyyaml pydantic typer rich
```

검증: `python -c "import torch; print(torch.cuda.is_available())"`를 실행해 True가 나오는지 확인한다. `python -c "from diff_gaussian_rasterization import GaussianRasterizer"`로 3DGS 컴파일이 되는지 확인한다.

작업 1.2: ScanNet → COLMAP 변환기 (src/data_utils/scannet_to_colmap.py)

추출된 ScanNet 장면 데이터를 COLMAP 이진 형식으로 변환한다.

입력 디렉터리 구조:

```

scannet_raw/scene0000_00/
├─ color/          # 0.jpg, 1.jpg, ...
├─ depth/          # 0.png, 1.png, ... (mm 단위 16비트 깊이)
├─ pose/           # 0.txt, 1.txt, ... (4x4 카메라-투-월드 행렬)
└─ intrinsic/
    └─ intrinsic_depth.txt # 4x4 내부 파라미터 행렬

```

출력 디렉터리 구조:

```

scannet_colmap/scene0000_00/
├─ images/         # RGB 이미지 심링크 또는 복사
└─ sparse/0/
    ├─ cameras.bin # 단일 PINHOLE 카메라 모델
    ├─ images.bin  # 이미지별 쿼터니언 + 변환
    └─ points3D.bin # 깊이 역투영으로 만든 초기 희소 포인트 클라우드

```

구현 시 핵심:

1. 4x4 내부 행렬을 읽어 COLMAP PINHOLE 모델용 f_x , f_y , c_x , c_y 를 추출한다.
2. 각 pose 파일의 4x4 카메라-투-월드 행렬을 읽어 역으로 월드-투-카메라를 구하고, COLMAP 관련에 맞게 쿼터니언(w, x, y, z)과 변환(t_x, t_y, t_z)으로 분해한다.
3. 일부 프레임(예: 10번째마다)에서 매 N번째(예: 20번째) 깊이 픽셀을 역투영해 초기 희소 포인트 클라우드를 만든다. 3DGS 초기화에 유리하다.
4. COLMAP Python 스크립트의 `read_write_model` 유틸리티 또는 `pycolmap`으로 이진 파일을 쓴다.

검증: 출력을 `pycolmap` 또는 3DGS 데이터 로더로 불러와 오류가 없는지 확인한다.

작업 1.3: 3DGS 학습 래퍼 (`src/reconstruction/train_3dgs.py`)

적절한 인자로 3DGS 학습 스크립트를 호출하는 Python 래퍼를 작성한다.

```

import subprocess
import os

def train_scene(data_path: str, output_path: str, iterations: int = 30000):
    """COLMAP 형식 장면에서 3DGS를 학습한다."""
    cmd = [
        "python", "submodules/gaussian-splatting/train.py",
        "-s", data_path,
        "-m", output_path,
        "--iterations", str(iterations),
    ]
    subprocess.run(cmd, check=True)

```

```
# 출력 검증
ply_path = os.path.join(output_path, "point_cloud", f"iteration_{iterations}",
                        "point_cloud.ply")
assert os.path.exists(ply_path), f"Training failed: {ply_path} not found"
return ply_path
```

검증: 일반적인 ScanNet 장면에서 출력 point_cloud.ply에 100,000개 이상의 Gaussian이 있어야 한다. render.py로 몇 개 시점을 렌더해 육안으로 확인한다.

작업 1.4: 뷰 렌더러 (src/reconstruction/render_views.py)

학습된 3DGS 모델에서 임의의 카메라 시점으로 RGB 이미지와 깊이 맵을 렌더하는 유틸리티를 작성한다. 분할 모듈에서 사용한다.

뷰당 출력:

- rgb_{view_id}.png (H x W x 3, uint8)
- depth_{view_id}.npy (H x W, float32, 장면 단위)
- alpha_{view_id}.png (H x W, uint8, 불투명도 맵)

2단계: Grounded SAM 2를 이용한 의미론적 분할

목표: 텍스트 쿼리와 카메라 뷰가 주어지면, 목표 객체에 해당하는 3D Gaussian을 식별한다.

기간: 3주차

작업 2.1: 텍스트 → 2D 마스크 (src/segmentation/text_to_mask.py)

Grounded SAM 2 파이프라인을 재사용 가능한 Python 함수로 감싼다.

```
def text_to_mask(image_path: str, text_prompt: str,
                 box_threshold: float = 0.3,
                 text_threshold: float = 0.25) -> np.ndarray:
    """
    Args:
        image_path: 렌더된 RGB 이미지 경로.
        text_prompt: 객체 설명 (예: "the wooden chair").
        box_threshold: Grounding DINO 검출 신뢰도 임계값.
        text_threshold: Grounding DINO 텍스트 유사도 임계값.

    Returns:
        mask: 이진 마스크 (H, W), dtype bool인 np.ndarray.
    """
```

구현 순서:

1. Grounding DINO 모델 로컬 체크포인트(submodules/Grounded-SAM-2/gdino_checkpoints/groundingdino_swinb_cogcoor.pth)를 로드한다.
2. 텍스트 프롬프트로 Grounding DINO를 실행해 바운딩 박스를 구한다.
3. SAM 2 이미지 예측기 초기화(체크포인트: submodules/Grounded-SAM-2/checkpoints/sam2.1_hiera_large.pt).
4. 바운딩 박스를 SAM 2에 박스 프롬프트로 전달한다.
5. 가장 신뢰도 높은 마스크를 반환한다.

메모리: 추론 후 모델을 명시적으로 삭제하고 `torch.cuda.empty_cache()`를 호출한다. 3·4단계에서는 이 모델이 필요하지 않다.

작업 2.2: 2D 마스크 → 3D Gaussian 인덱스 (src/segmentation/mask_to_gaussians.py)

```
def mask_to_gaussian_indices(
    mask: np.ndarray,          # (H, W) bool
    depth_map: np.ndarray,     # (H, W) float32
    camera_intrinsics: np.ndarray, # (3, 3)
    camera_extrinsics: np.ndarray, # (4, 4) world-to-camera
    gaussian_positions: np.ndarray, # (N, 3) 모든 Gaussian의 xyz
    distance_threshold: float = 0.05
) -> np.ndarray:
    """
    Returns:
        indices: Gaussian 배열에 대한 정수 인덱스 1D 배열.
    """
```

구현 순서:

1. 마스크가 True인 각 픽셀을 깊이와 카메라 파라미터로 3D에 역투영한다.
2. `gaussian_positions`로 KD-tree를 만든다.
3. 각 역투영 3D 점에 대해 `distance_threshold` 안에서 가장 가까운 Gaussian을 조회한다.
4. 모든 고유 Gaussian 인덱스를 모은다.

작업 2.3: 다시점 합의 (src/segmentation/multi_view_consensus.py)

견고성을 위해 여러 카메라 시점에서 작업 2.1·2.2를 실행하고, 나온 Gaussian 인덱스 집합의 교집합(또는 다수결)을 취한다.

입력: 카메라 뷰 인덱스 목록(예: `[0, 10, 20, 30]`).

출력: N개 뷰 중 적어도 2개에서 등장하는 최종 Gaussian 인덱스 집합.

3단계: SDS 기반 객체 생성 및 배치

목표: 텍스트로 새 3D Gaussian 객체를 생성하고 장면에 배치한다.

기간: 4-5주

작업 3.1: SDS 객체 생성기 (src/generation/sds_generator.py)

DreamGaussian의 텍스트-투-3D 파이프라인을 감싼다.

```
def generate_object(text_prompt: str, output_dir: str,
                   num_steps: int = 500) -> str:
    """
    Args:
        text_prompt: 객체 설명 (예: "a red rubber duck").
        output_dir: 생성된 .ply 파일을 저장할 디렉터리.
        num_steps: SDS 최적화 스텝 수.

    Returns:
        ply_path: 생성된 객체 PLY 파일 경로.
    """
```

주의점:

1. DreamGaussian은 원점 중심 객체를 생성한다. 출력 PLY는 기본 장면과 동일한 좌표계·스케일이어야 한다.
2. 생성 후 src/generation/object_rescaler.py로 장면 대비 객체 크기를 정규화한다(예: "고무 오리"는 ~0.15m 수준이어야지 2m이면 안 됨).
3. DreamGaussian SDS 품질이 부족하면 두 단계 접근(SDS 최적화 → 메시 추출 및 텍스처 정제)을 고려한다.

대안: SDS 결과가 나쁠 때(얼굴 여러 개, 흐린 텍스처) InstantMesh나 TripoSR 등으로 먼저 메시를 만든 뒤 gaussian-splatting 초기화로 Gaussian으로 변환한다.

작업 3.2: 표면 인지 배치 (src/placement/surface_aware.py)

```
def find_surface_height(scene_gaussians: np.ndarray,
                       x: float, y: float,
                       search_radius: float = 0.1) -> float:
    """
    (x, y)에서 가장 가까운 표면의 z 좌표를 찾는다.

    과정:
    1. xy 평면에서 (x, y)의 search_radius 안에 있는 Gaussian만 남긴다.
    2. 필터된 Gaussian 중 불투명도가 높음(> 0.5) 것들 중 z가 가장 큰 것을 찾는다(표면 위쪽).
```

3. 그 z 를 배치 높이로 반환한다.
"""

작업 3.3: PLY 병합 (src/placement/ply_merger.py)

```
def merge_ply_files(base_ply: str, object_ply: str,
                    translation: np.ndarray, # (3,) 오프셋
                    output_ply: str) -> str:
    """
    두 3DGS PLY 파일을 병합한다.

    중요: 두 PLY의 정점 속성 스키마가 동일해야 한다.
    3DGS PLY에는 x, y, z, nx, ny, nz, f_dc_0..2, f_rest_0..44,
    opacity, scale_0..2, rot_0..3가 포함된다.

    단계:
    1. plyfile.PlyData로 두 PLY를 로드한다.
    2. 객체의 x, y, z에 translation을 적용한다.
    3. 속성 목록이 같은지 검증한다.
    4. 정점 배열을 이어붙인다.
    5. 새 PLY로 저장한다.
    """
```

4단계: PhysGaussian을 이용한 물리 시뮬레이션

목표: 선택되거나 생성된 Gaussian에 MPM 기반 물리 효과를 적용한다.

기간: 6-7주

작업 4.1: 재질 프리셋 (src/physics/material_presets.py)

PhysGaussian이 기대하는 파라미터에 맞는 재질 설정 딕셔너리를 정의한다.

```
MATERIAL_PRESETS = {
    "jelly": {
        "material": "jelly",
        "E": 2e4,          # 영률 (Pa)
        "nu": 0.3,         # 포아송비
        "density": 1000,   # kg/m^3
    },
    "metal": {
        "material": "metal",
        "E": 5e5,
        "nu": 0.35,
        "density": 7800,
    },
}
```

```

    "sand": {
        "material": "sand",
        "E": 1e4,
        "nu": 0.2,
        "density": 1500,
    },
    "foam": {
        "material": "foam",
        "E": 1e3,
        "nu": 0.25,
        "density": 200,
    },
}

```

작업 4.2: 설정 생성기 (src/physics/config_generator.py)

```

def generate_phys_config(
    gaussian_indices: np.ndarray,
    gaussian_positions: np.ndarray,
    material_name: str,
    output_path: str,
    n_grid: int = 100,
    frame_num: int = 120,
    frame_dt: float = 0.01,
    substep_dt: float = 4e-4,
    gravity: float = -9.8,
    camera_index: int = 0
) -> str:
    """
    PhysGaussian 호환 JSON 설정 파일을 생성한다.

    설정에는 다음이 포함되어야 한다:
    - 전처리: opacity_threshold, rotation, sim_area
    - 시뮬레이션: 재질 파라미터, 그리드, 중력, 경계 조건
    - 내보내기: 프레임 수, 카메라 인덱스
    """

```

세부: PhysGaussian의 sim_area는 목표 Gaussian의 바운딩 박스에 20% 여유를 두어 계산한다. boundary_conditions에는 시뮬레이션 도메인 밖으로 입자가 나가지 않도록 bounding_box 유형을 넣는다.

작업 4.3: MPM 실행기 (src/physics/mpm_runner.py)

```

def run_simulation(model_path: str, config_path: str,
                  output_path: str) -> str:
    """
    PhysGaussian 시뮬레이션을 실행하고 출력 비디오 경로를 반환한다.
    """

```

```

cmd = [
    "python", "submodules/PhysGaussian/gs_simulation.py",
    "--model_path", model_path,
    "--output_path", output_path,
    "--config", config_path,
    "--render_img",
    "--compile_video",
]
subprocess.run(cmd, check=True)
video_path = os.path.join(output_path, "video.mp4")
assert os.path.exists(video_path)
return video_path

```

5단계: 평가 및 최종 산출물

목표: 정량·정성 평가 및 최종 보고서.

기간: 8주차

작업 5.1: 정량 지표 (src/evaluation/metrics.py)

편집된 장면의 렌더 뷰를 정답 또는 기준선과 비교하는 아래 지표를 구현한다.

지표	라이브러리	용도
PSNR	skimage.metrics.peak_signal_noise_ratio	픽셀 수준 재구성 품질
SSIM	skimage.metrics.structural_similarity	구조적 유사도
LPIPS	lpips (pip 패키지)	지각적 유사도
FPS	직접 타이머	실시간 렌더링 성능

작업 5.2: 데모 비디오 생성

다음을 보여주는 정제된 데모 영상을 만드는 스크립트를 작성한다:

1. 원본 재구성 장면(궤도 카메라).
2. 모드 A: 텍스트 프롬프트 → 객체 등장 → 물리 시뮬레이션 재생.
3. 모드 B: 텍스트 프롬프트 → 객체 하이라이트 → 물리 효과 적용.

ffmpeg으로 클립을 이어붙이고 텍스트 오버레이를 쓴다.

작업 5.3: 애플레이션

시간이 되면 Feature Splatting의 LERF 기반과 비교해 SAM 2 분할 정확도를 측정한다. 테스트 객체 5개에서 분할 마스크 IoU를 잰다.

3. 전역 설정 스키마

메인 파이프라인 설정(configs/pipeline_config.yaml)은 아래 스키마를 따른다:

```
# 장면 설정
scene:
  scannet_scene_id: "scene0000_00"
  data_root: "data/scannet_raw"
  colmap_output: "data/scannet_colmap"
  model_output: "output/base_scene"
  training_iterations: 30000

# 분할 설정
segmentation:
  grounding_dino_checkpoint: "submodules/Grounded-SAM-
    2/gdino_checkpoints/groundingdino_swinb_cogcoor.pth"
  sam2_checkpoint: "submodules/Grounded-SAM-2/checkpoints/sam2.1_hiera_large.pt"
  sam2_config: "submodules/Grounded-SAM-2/sam2/configs/sam2.1/sam2.1_hiera_l.yaml"
  box_threshold: 0.3
  text_threshold: 0.25
  multi_view_count: 4
  consensus_min_votes: 2

# 생성 설정 (모드 A)
generation:
  stable_diffusion_model: "stabilityai/stable-diffusion-2-1-base"
  sds_steps: 500
  guidance_scale: 100.0
  object_scale_factor: 0.1 # 장면 바운딩 박스 대비 스케일

# 물리 설정
physics:
  n_grid: 100
  substep_dt: 4e-4
  frame_dt: 0.01
  frame_num: 120
  gravity: -9.8

# 평가
evaluation:
  test_views: [0, 5, 10, 15, 20]
  lpips_net: "alex"
```

4. Cursor AI를 위한 구현 시 주의사항

4.1 VRAM 관리 전략

네 가지 주요 모델은 일반적인 24GB GPU에서 동시에 올릴 수 없다. 순차 로드/언로드 패턴을 구현해야 한다:

```
# pipeline.py 예시 패턴
def run_pipeline(config):
    # 1단계: 3DGS 학습 (약 8GB VRAM)
    train_scene(...)
    torch.cuda.empty_cache()

    # 2단계: 분할 (SAM2 + GDINO 약 12GB VRAM)
    indices = run_segmentation(...)
    del sam2_model, gdino_model
    torch.cuda.empty_cache()
    gc.collect()

    # 3단계: SDS 생성 (SD 약 10GB VRAM)
    object_ply = generate_object(...)
    del sd_pipeline
    torch.cuda.empty_cache()
    gc.collect()

    # 4단계: 물리 (MPM 약 6GB VRAM)
    run_simulation(...)
```

4.2 서브모듈 간 PLY 호환성

PhysGaussian은 자체 gaussian-splatting 포크를 서브모듈로 쓴다. PLY 형식이 메인 gaussian-splatting 저장소와 약간 다를 수 있다(예: SH 계수 개수). PLY를 병합하기 전에:

1. 기본 장면 PLY와 PhysGaussian이 기대하는 PLY 형식을 모두 로드한다.
2. 정점 속성 스키마를 비교한다.
3. 다르면 SH 계수를 패딩하거나 잘라서 변환하는 함수를 작성한다.

4.3 좌표계 정렬

DreamGaussian은 정규화된 좌표 공간(대략 $[-1, 1]^3$)에서 객체를 생성하고, ScanNet 장면은 미터 단위 좌표를 쓴다. object_rescaler.py는:

1. 기본 장면의 바운딩 박스를 계산한다.

2. 생성 객체를 물리적으로 그럴듯한 크기로 스케일한다(object_scale_factor로 설정).
3. PhysGaussian이 기대하는 동일한 회전/이동을 적용한다(장면은 축에 정렬되어 있어야 한다).

4.4 PhysGaussian 장면 준비

PhysGaussian은 장면이 축에 정렬되어 있어야 한다(예: 바닥이 xy 평면과 평행). ScanNet 장면은 임의 방향일 수 있다. 따라서:

1. Gaussian 위치에 RANSAC으로 지배적인 바닥 평면을 찾는다.
2. 그 평면이 $z=0$ 에 오도록 하는 회전 행렬을 계산한다.
3. 이 회전을 PhysGaussian 설정 JSON(rotation_degree, rotation_axis)에 저장한다.

5. 테스트 및 검증 체크리스트

각 단계마다 통과/실패 기준이 있다. 모두 충족하기 전에는 다음 단계로 넘어가지 않는다.

단계	테스트	통과 기준
1	setup.sh 무오류 완료	모든 import 성공
1	COLMAP 변환	pycolmap.Reconstruction() 오류 없이 로드
1	3DGS 학습	point_cloud.ply 존재, 정점 100K 초과
1	뷰 렌더링	렌더 이미지가 육안으로 인식 가능
2	텍스트→마스크	마스크 오버레이가 목표 객체를 올바르게 강조
2	마스크→Gaussian	선택 Gaussian을 3D로 시각화 시 목표 형상과 일치
3	SDS 생성	텍스트 프롬프트와 일치하는 객체로 인식 가능
3	배치	병합 PLY가 올바르게 렌더, 표면 위에 놓이고 떠 있거나 클리핑 없음
4	물리 시뮬레이션	10프레임 테스트 시 OOM 없음, 변형이 그럴듯함
4	전체 시뮬레이션	120프레임 비디오가 부드러운 물리 애니메이션
5	지표	PSNR/SSIM/LPIPS가 계산되어 CSV에 저장

6. 산출물 요약

완료 시 다음 산출물이 있어야 한다:

산출물	형식	설명
동작하는 파이프라인 코드	Python	모드 A·B 종단간
데모 비디오 (모드 A)	.mp4	텍스트 → 객체 생성 → 배치 → 물리
데모 비디오 (모드 B)	.mp4	텍스트 → 객체 선택 → 물리 효과
정량 결과	.csv	뷰별 PSNR, SSIM, LPIPS, FPS
기술 보고서	.pdf	방법, 결과, 애플리케이션
설정 파일	.yaml + .json	재현 가능한 실험 설정